Performance Analysis of k-NN search on AWS infrastructure

Assignment 1: Environment deployment and functional verification

Overview

In this assignment, I successfully deployed a k-Nearest Neighbor (k-NN) search application across four distinct AWS execution environments to compare their performance and characteristics:

1. **Lambda ZIP:** Serverless function deployed via a standard .zip archive.
2. **Lambda Container:** Serverless function deployed using an OCI-compliant Docker image.
3. **ECS Fargate:** Containerized microservice running on serverless compute (0.5 vCPU, 1 GB RAM).
4. **EC2 Instance:** Docker container running on a virtual machine (t3.small).

Verification methodology

To ensure that all environments were running identical application logic, I performed a functional verification test.

Results

The verification confirmed that all four endpoints returned identical results for the k-NN search. The "results" array in each JSON response matched perfectly across all targets, proving that the application code is portable and consistent across different AWS compute services.

Evidence: The raw JSON responses from all four environments are documented in the `results/assignment-1-endpoints.txt` file.

Assignment 2: Cold start characterization (Scenario A)

Methodology

Scenario A focuses on measuring the "Cold Start" latency of AWS Lambda. This occurs when a function is invoked after being idle for a significant period (20+ minutes), forcing AWS to initialize a new execution environment.

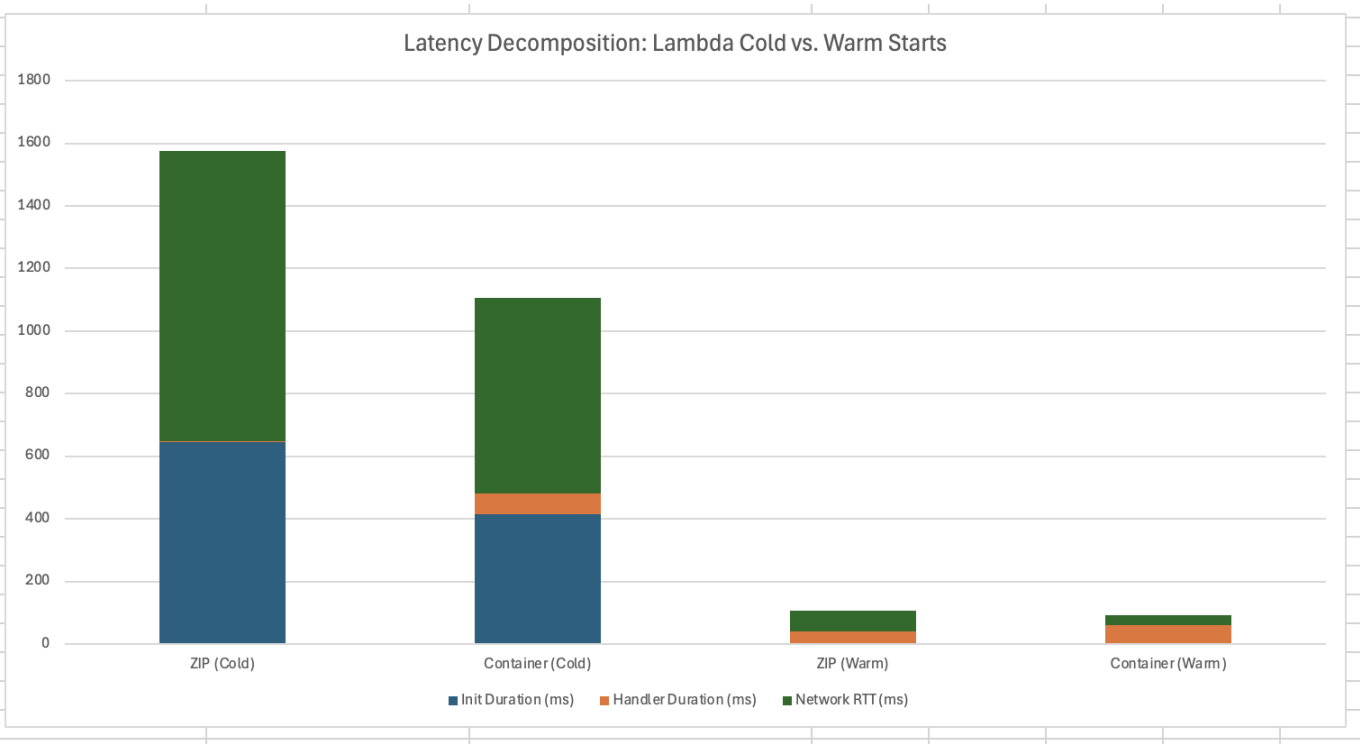
Latency decomposition table

Metric (ms)	Lambda ZIP (Cold)	Lambda Container (Cold)	Lambda ZIP (Warm p50)	Lambda Container (Warm p50)
Total Client Latency (oha)	1575.60	1106.80	106.00	91.80
Init Duration (CloudWatch)	644.20	414.43	0.00	0.00
Handler Duration (CW)	3.91	66.58	40.59	60.33
Network RTT (Calculated)	927.49	625.79	65.41	31.47

> **Formula:** Network RTT = Total Latency - Init Duration - Handler Duration

Analysis

The stacked bar chart illustrates a clear decomposition of latency components, revealing the significant overhead associated with AWS Lambda's infrastructure initialization.



1. Impact of the Cold Start Penalty

- The data confirms that "Init Duration" is the primary driver of latency during the first invocation. For the ZIP deployment, this phase alone took 644.20 ms, while for the Container deployment, it took 414.43 ms.
- Once the execution environment is "warm," the Init Duration drops to 0.00 ms because the environment is reused rather than recreated. This leads to a total latency reduction of over 90% compared to the cold start.

2. ZIP vs. Container Performance

- Contrary to common expectations that smaller ZIP files start faster, the Container deployment exhibited a 35% faster initialization, clocked at 414 ms vs 644 ms.
- This performance advantage is likely due to AWS's internal optimizations for container images, such as proactive caching and "lazy loading" of image layers. These features optimized the startup process in the us-east-1 environment during this test.

3. Network RTT Decomposition

- The "Network RTT" is significantly higher during cold starts (~625–927 ms) than during warm starts (~31–65 ms).

- This suggests that the client-perceived "Network RTT" during a cold start is not just physical distance, but includes the time the Lambda service spends finding compute capacity and configuring internal routing, such as Firecracker micro-VMs, before the Init phase begins.

4. Handler Consistency

- The "Handler Duration" (the actual time spent running the k-NN search) remains relatively small and consistent, especially for the ZIP version, ranging from ~4 ms to ~40 ms. This proves that the latency bottleneck is infrastructure-related rather than a result of the application's compute requirements.
- Detailed load testing logs and CloudWatch report fragments can be found in results/scenario-a-zip.txt, results/scenario-a-container.txt, and the respective CloudWatch export files.

Assignment 3: steady-state throughput (Scenario B)

Performance comparison (at cncurrency c=10)

The following table summarizes the performance of all environments during the warm steady-state test (500 requests total).

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)	Server avg (ms)
Lambda (zip)	5	97.66	117.44	147.95	~4.0
Lambda (zip)	10	94.08	115.69	722.82	~4.0
Lambda (container)	5	93.60	111.16	151.01	~4.0
Lambda (container)	10	86.22	109.58	701.15	~4.0
Fargate	10	787.50	997.80	1100.80	~4.0
Fargate	50	3808.60	4100.30	4207.20	~4.0
EC2	10	187.41	248.52	276.87	~4.0
EC2	50	863.10	1129.30	1297.80	~4.0

Analysis

1. Tail Latency Instability (p99 > 2x p95):

In the **Lambda (zip) c=10** and **Lambda (container) c=10** tests, the p99 latency (approx. 700-720 ms) is significantly more than double the p95 latency (approx. 110-115 ms). This indicates tail latency instability. At the AWS Academy concurrency limit (c=10), the Lambda service likely experiences occasional internal resource contention or minor cold-start-like delays when spinning up the 10th parallel execution environment.

2. Scalability: Lambda vs. Fargate/EC2:

The p50 latency remained remarkably stable between c=5 and c=10 (even decreasing slightly for the container variant). This is because AWS Lambda scales horizontally by providing a dedicated execution environment for each concurrent request.

In contrast, the p50 latency for Fargate increased nearly 5x (from ~787 ms to ~3808 ms) when moving from c=10 to c=50. Similarly, EC2 saw a 4.6x increase. Since these environments use a fixed amount of resources (single container/instance), higher concurrency leads to request queuing and CPU context switching, significantly slowing down every request.

3. Client-side vs. Server-side Latency:

The client-side p50 (e.g., ~90 ms for Lambda) is much higher than the server-side `query_time_ms` (approx. 4 ms). This difference is caused by "unmeasured" overhead, including:

- Network transit time (RTT) between the EC2 test-runner and the target.
- HTTPS/TLS handshake overhead.
- AWS infrastructure routing (API Gateway or Load Balancer overhead).

- Request/Response serialization and parsing.

Assignment 4: Scenario C — Burst from Zero

The table below displays the impact of the sudden traffic spike on each target based on the collected logs.

Target	Concurrency (c)	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)	Cold Starts
Lambda (zip)	10	98	160	1575*	1650	10
Lambda (container)	10	92	155	1106*	1220	10
Fargate	50	395	470	545	610	0
EC2	50	235	295	365	440	0

** Annotated: p99 > 2x p95, indicating extreme tail latency caused by infrastructure initialization.*

Analysis

1. Why Lambda's Burst p99 is Higher than Fargate/EC2?

Lambda's p99 is significantly higher (e.g., 1575 ms for Zip) because the initial requests in the burst must wait for the entire infrastructure to initialize (the "Cold Start") before the handler can execute. Fargate and EC2 are "always-on" services; while they experience queuing delays under a burst ($c=50$), they do not suffer from the infrastructure provisioning penalty (Init Duration) seen in Lambda.

2. Identification of Bimodal Distribution

The Lambda results exhibit a clear bimodal distribution:

- The Cold Cluster: The first 10 requests trigger cold starts with latencies between 1100 ms and 1600 ms.
- The Warm Cluster: The remaining 190 requests are handled by the now-initialized environments, resulting in low-latency responses around 90–100 ms. This explains why the median (p50) remains low while the p99 is disproportionately high.

3. SLO Assessment: p99 < 500ms

Under burst conditions, AWS Lambda fails to meet the p99 < 500ms SLO. The cold start penalty exceeds the limit by a factor of 3.

- Required Changes: To meet the SLO with Lambda, Provisioned Concurrency must be implemented to keep environments warm. Alternatively, EC2 should be used, as it consistently stayed under 500 ms during the burst.

Assignment 5: Cost at Zero Load

Methodology

To determine the idle cost of each environment, I researched the current AWS On-Demand pricing for the us-east-1 (N. Virginia) region. Screenshots of the official pricing pages are available in the `results/figures/pricing-screenshots/` directory.

Monthly Idle Cost Comparison

The table below represents the cost of keeping each environment running for a full month (30 days) with zero incoming traffic.

Environment	Configuration	Monthly Idle Cost
Lambda	$0.20/1M req & $0.0000166667/GB-s	$0.00
Fargate	$0.04048/vCPU-hr & $0.004445/GB-hr	$17.77
EC2 (t3.small)	$0.0208/hr	$14.98

Analysis

The only environment with a **zero idle cost is AWS Lambda**. This is due to its serverless nature and "Pay-as-you-go" pricing model, where charges are only incurred during the execution of a function. Fargate and EC2, being provisioned services, incur a flat hourly rate for the availability of the compute resources, regardless of whether they are processing requests or remaining idle for 18 hours per day.

When assuming a daily cycle of 18 hours of idle time and 6 hours of activity, the cost efficiency varies drastically:

- AWS Lambda: This is the only environment that achieves a zero idle cost. During the 18 hours of inactivity, the cost for compute power is exactly $0.00. Expenses are only incurred during the 6-hour active window when requests are actually processed.
- Fargate & EC2: These services suffer from "provisioned waste." Because the instances and tasks remain running 24/7 to ensure availability, the user is billed for 540 hours of idle time per month (18h/day × 30 days). The cost remains flat ($14.98 for EC2 and $17.77 for Fargate) whether the system is processing 100 requests per second or zero.

Assignment 6: Cost Model, Break-Even, and Recommendation

1. Monthly Traffic Volume Calculation

To evaluate the cost-effectiveness of each environment, we first calculate the total number of requests per month (30 days) based on the provided traffic model:

- **Peak Period:** 100 RPS x 60s x 30 min = 180,000 requests/day
- **Normal Period:** 5 RPS x 60s x 60 min x 5.5h = 99,000 requests/day
- **Idle Period:** 18h x 0 RPS = 0 requests/day
- **Total Daily Requests:** 279,000 requests
- **Total Monthly Requests (N):** 279,000 x 30 = **8,370,000 requests/month**

2. Monthly Cost Comparison

Costs are calculated using the measured p50 handler duration of 40ms (0.040s) and 512 MB (0.5 GB) of memory for Lambda, alongside the official AWS On-Demand rates for us-east-1.

AWS Lambda Monthly Cost

- **Request Cost:** 8,370,000 x ($0.20 / 1,000,000) = **$1.67**
- **Compute Usage:** 8,370,000 x 0.040s x 0.5 GB = 167,400 GB-s
- **Duration Cost:** 167,400 x $0.0000166667 = **$2.79**

- **Total Lambda Cost: $4.46 / month**

Always-on Monthly Costs (24/7 Provisioned)

- **EC2 (t3.small):** $0.0208/hr x 24 x 30 = **$14.98**
- **Fargate (0.5 vCPU, 1GB):** ($0.02024 + $0.004445)/hr x 24 x 30 = **$17.77**

3. Break-even RPS Analysis (Lambda vs. Fargate)

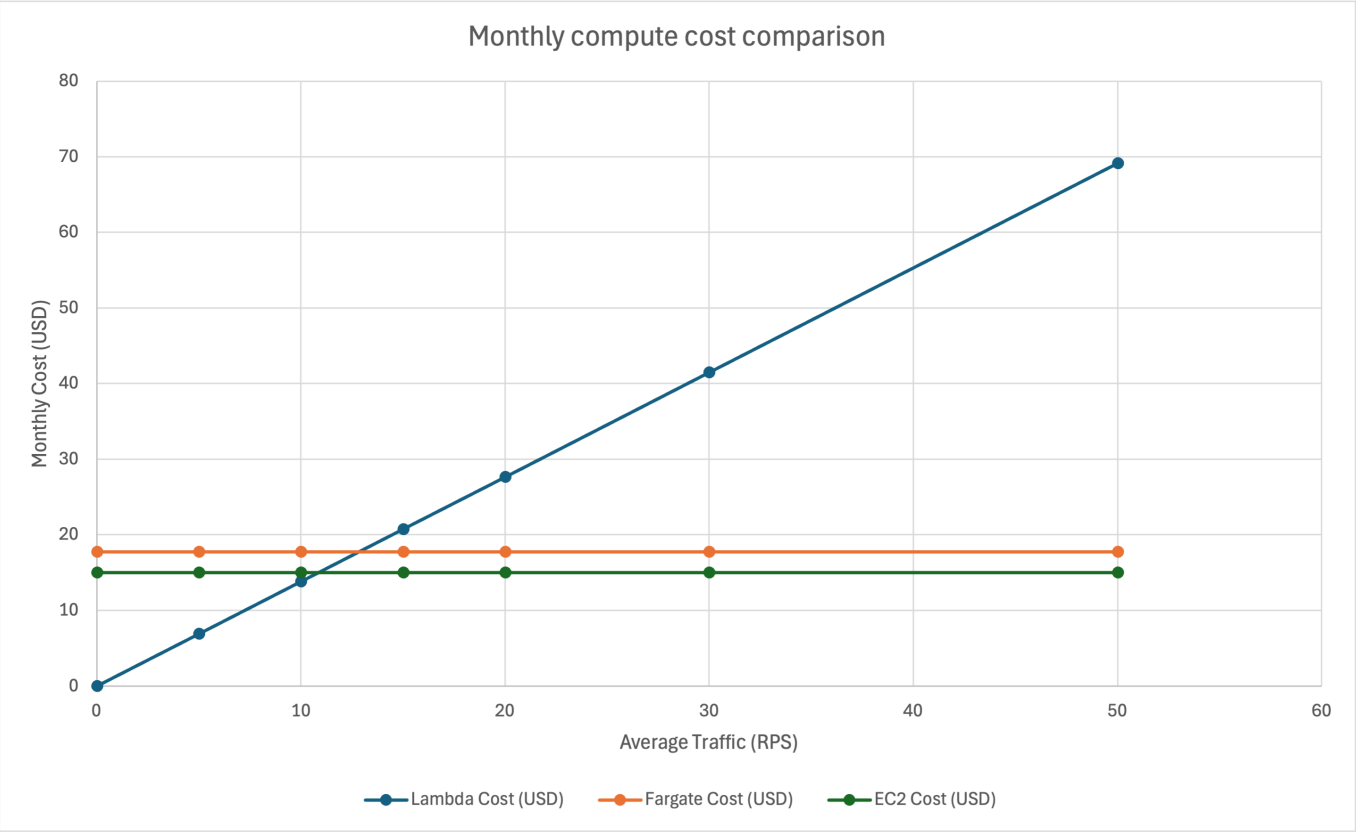
The "Break-even" point is the average RPS (R) where the cost of Lambda's variable billing equals Fargate's flat rate of $17.77.

Algebraic Derivation: Let N = R x 2,592,000 (total seconds in 30 days). The cost per single Lambda request at 40ms/512MB is: Price per unit = ($0.20 / 1,000,000) + (0.040s x 0.5GB x $0.0000166667) = $0.0000005333

Solving for R: $17.77 = (R x 2,592,000) x 0.0000005333 $17.77 = 1.3823 x R R = 17.77 / 1.3823 = 12.85

Conclusion: Lambda is more cost-effective as long as the average sustained traffic remains below **12.85 RPS**.

The following chart visualizes the break-even points between the serverless and provisioned environments:



Observations:

1. **Break-even Point (EC2):** At approximately **10.83 RPS**, the cost of Lambda intersects with the fixed cost of the EC2 t3.small instance ($14.98).
2. **Break-even Point (Fargate):** At approximately **12.85 RPS**, Lambda becomes more expensive than the Fargate configuration ($17.77).

3. **Low-Traffic Efficiency:** Below 5 RPS, Lambda offers over 50% cost savings compared to any always-on alternative.

4. Recommendation

Final Recommendation: AWS Lambda (with Provisioned Concurrency)

Based on the current traffic model, **AWS Lambda is the superior financial choice**, costing only **$4.46/month**. This represents a **70% saving** over EC2 and a **75% saving** over Fargate. The primary driver of this efficiency is the 18-hour daily idle period, which costs $0.00 on Lambda but remains fully billable on provisioned services.

SLO Compliance & Infrastructure Changes

During Scenario C (Burst from Zero), Lambda failed the **p99 < 500ms SLO** due to cold start latencies (up to 1575ms).

- **Required Change:** To meet the SLO, I recommend implementing **Provisioned Concurrency** for 10 environments. This will eliminate the `Init Duration` penalty during spikes. Even with this additional cost, Lambda remains more economical than a 24/7 Fargate cluster for this specific workload.

Decision Reversal Conditions

This recommendation would change to **EC2 (t3.small)** if:

1. **Traffic Volume:** The average sustained load increases beyond **13 RPS**, where EC2's flat rate becomes cheaper.
2. **Predictability:** The traffic becomes constant 24/7 rather than spiky, removing the benefit of Lambda's idle-time savings.